API Security Risk Analysis Report

Modern SaaS Skill Assessment

Prepared For: Internal Analysis / Portfolio Project

Prepared By: Eddy Biegon

Date: March 2026

Table of Contents

1. Executive Summary 3
2. Scope and Methodology 4
2.1 Scope 4
2.2 Methodology 4
2.3 Tools Used 4
2.4 Limitations 4
3. Target API Overview 5
3.1 API Description 5
3.2 Available Resources 5
3.3 Expected Security Posture 5
4. Detailed Risk Findings 6
4.1 Open/Unauthenticated Endpoints 6
Business Impact 7
Remediation Steps 7
4.2 Broken Object Level Authorization (IDOR) 8
Business Impact 10
4.3 Excessive Data Exposure 11
Business Impact 12
Remediation Steps 12
4.4 Missing Rate Limiting 13
Evidence 14
Business Impact 14
Remediation Steps 15
5. Risk Summary Table 16
6. Conclusion 16
6.1 Summary of Findings 16

6.2 Key Recommendations 16

6.3 Final Risk Assessment 17

Appendix A: OWASP API Security Top 10 Reference 17

1. Executive Summary

This report presents the findings of a comprehensive API Security Risk Analysis conducted on jsonplaceholder.typicode.com, a publicly available REST API designed for testing and prototyping. The analysis was performed as a read-only assessment to identify common API security vulnerabilities based on the OWASP API Security Top 10 framework.

Key Findings Summary:

Risk Category	Severity	Finding
1. **Broken Authentication**	CRITICAL	Complete lack of authentication on all endpoints
2. **Lack of Resources & Rate Limiting**	CRITICAL	No rate limiting controls implemented
3. **Broken Object Level Authorization**	HIGH	Any resource accessible by ID without authorizatio
4. **Excessive Data Exposure**	MEDIUM	Sensitive user data returned unnecessarily

Overall Risk Assessment: The API exhibits **CRITICAL** security vulnerabilities that would be unacceptable in any production SaaS environment. While JSONPlaceholder is intentionally designed as an open testing API, this analysis demonstrates the security controls that must be implemented in real-world applications.

2. Scope and Methodology

2.1 Scope

Aspect	Details
Target API	https://jsonplaceholder.typicode.com
Assessment Type	Read-only security analysis
Testing Period	March 2026
Endpoints Tested	/posts, /users, /comments, /albums, /todos

2.2 Methodology

The assessment followed a systematic approach:

1. **Documentation Review**: Analyzed official API documentation to understand endpoints, expected responses, and any stated security controls.
2. **Endpoint Testing**: Performed read-only GET requests to all major endpoints using Postman.
3. **Risk Identification**: Mapped findings to OWASP API Security Top 10 categories.
4. **Evidence Collection**: Captured screenshots of requests, responses, and headers for documentation.
5. **Severity Assessment**: Classified risks as Critical, High, Medium, or Low based on potential business impact.

2.3 Tools Used

Tool	Purpose
Postman	API request testing and response inspection
Browser DevTools	Header analysis and additional validation
OWASP API Security Top 10	Risk classification framework

2.4 Limitations

This assessment was limited to read-only GET requests. No exploitation, brute force attacks, or denial-of-service testing was performed. The analysis focuses on identifying risks, not exploiting vulnerabilities.

---

3. Target API Overview

3.1 API Description

JSONPlaceholder is a free online REST API that provides fake data for testing and prototyping. It is widely used by developers to test applications before building production backends.

Base URL: https://jsonplaceholder.typicode.com

3.2 Available Resources

Resource	Endpoint	Number of Records
• **Posts**	/posts	100 posts
• **Comments**	/comments	500 comments
• **Albums**	/albums	100 albums
• **Photos**	/photos	5000 photos
• **Todos**	/todos	200 todos
• **Users**	/users	10 users

3.3 Expected Security Posture

As a public test API, JSONPlaceholder intentionally implements minimal security controls. However, this analysis evaluates it against production security standards to demonstrate vulnerabilities that must be addressed in real SaaS applications.

4. Detailed Risk Findings

4.1 Open/Unauthenticated Endpoints

Risk Identifier: AUTH-001

Field	Description
Affected Endpoints	ALL endpoints
OWASP Category	API2:2023 - Broken Authentication
Severity	CRITICAL
Likelihood	High
Impact	High

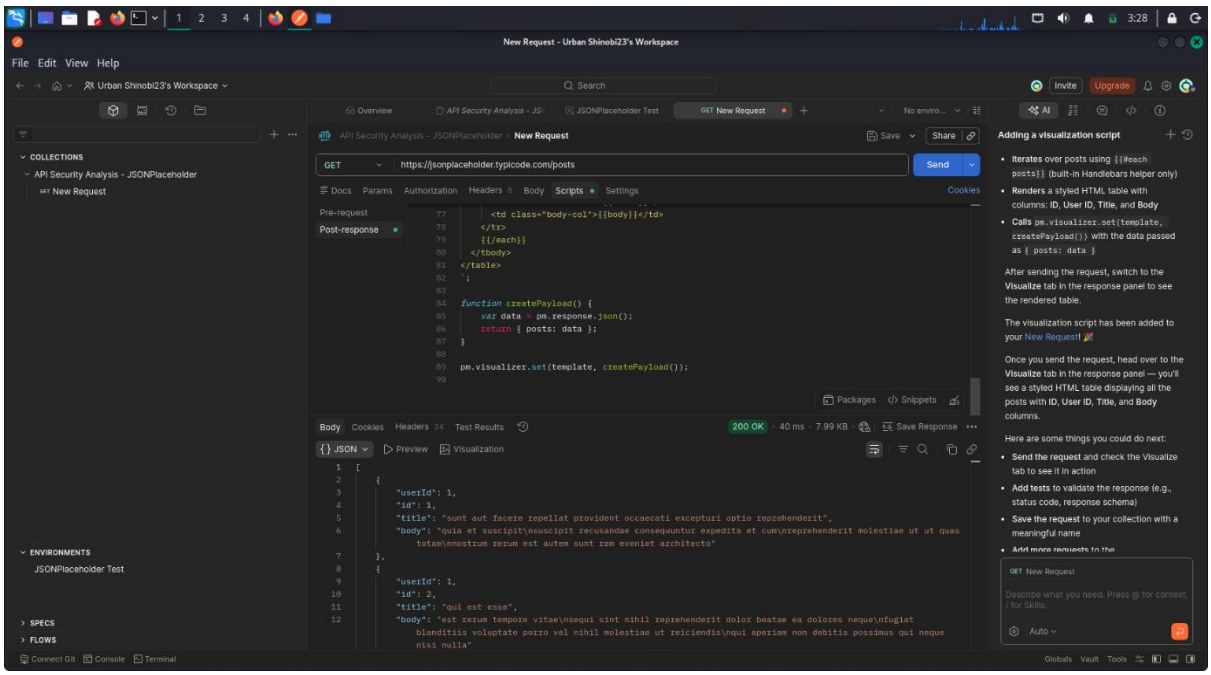
Description

The API implements no authentication mechanism whatsoever. All endpoints are completely open and accessible without any API keys, tokens, session cookies, or other forms of identity verification.

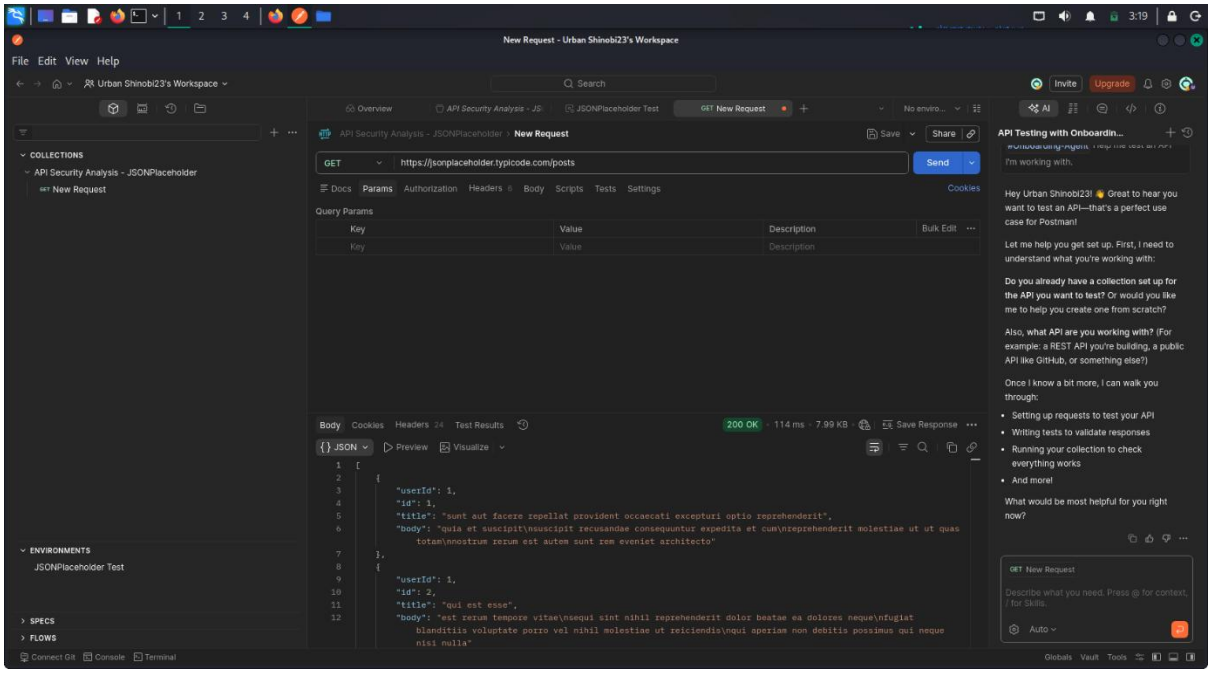
Test Performed

A GET request was sent to https://jsonplaceholder.typicode.com/posts with **no Authorization headers** and **no authentication credentials**.

Evidence



POST REQUEST 1



POST REQUEST 2

Business Impact

Impact Area	Description
Data Breach Risk	Anyone can access all API data without restriction
Accountability	No way to track who is accessing what data
Compliance	Violates GDPR, CCPA, HIPAA requirements for access control
Abuse Potential	Attackers can freely scrape all data without identification

Remediation Steps

1. **Implement OAuth 2.0**: Deploy industry-standard OAuth 2.0 for authorization
    - Use Authorization Code flow for first-party applications
    - Use Client Credentials flow for server-to-server integration
2. **JWT Implementation**: If using token-based authentication:
    - Require valid JWT in Authorization: Bearer <token> header
    - Implement proper token expiration (short-lived access tokens)
    - Use refresh tokens for session management
3. **API Key Management**: For service-to-service communication:
    - Issue unique API keys to each client
    - Validate keys on every request

- Implement key rotation policies

4. **Multi-Factor Authentication**: For sensitive operations, require MFA

4.2 Broken Object Level Authorization (IDOR)

Risk Identifier: AUTHZ-001

Field	Description
Affected Endpoints	/posts/{id}, /users/{id}, /comments/{id}, /albums/{id}
OWASP Category	API1:2023 - Broken Object Level Authorization
Severity	HIGH
Likelihood	High
Impact	High

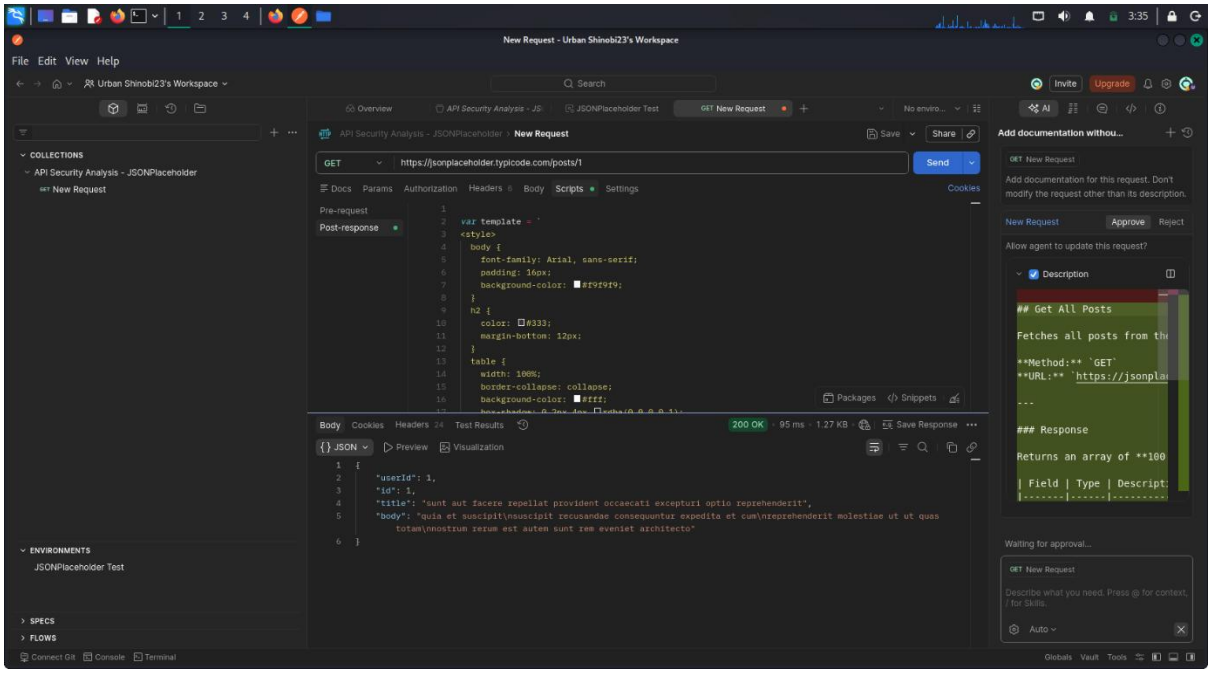
Description

The API allows access to any object by simply changing its ID in the URL, with no verification that the requester is authorized to view that specific object. This vulnerability, known as Insecure Direct Object Reference (IDOR), allows attackers to enumerate and access all records in the system.

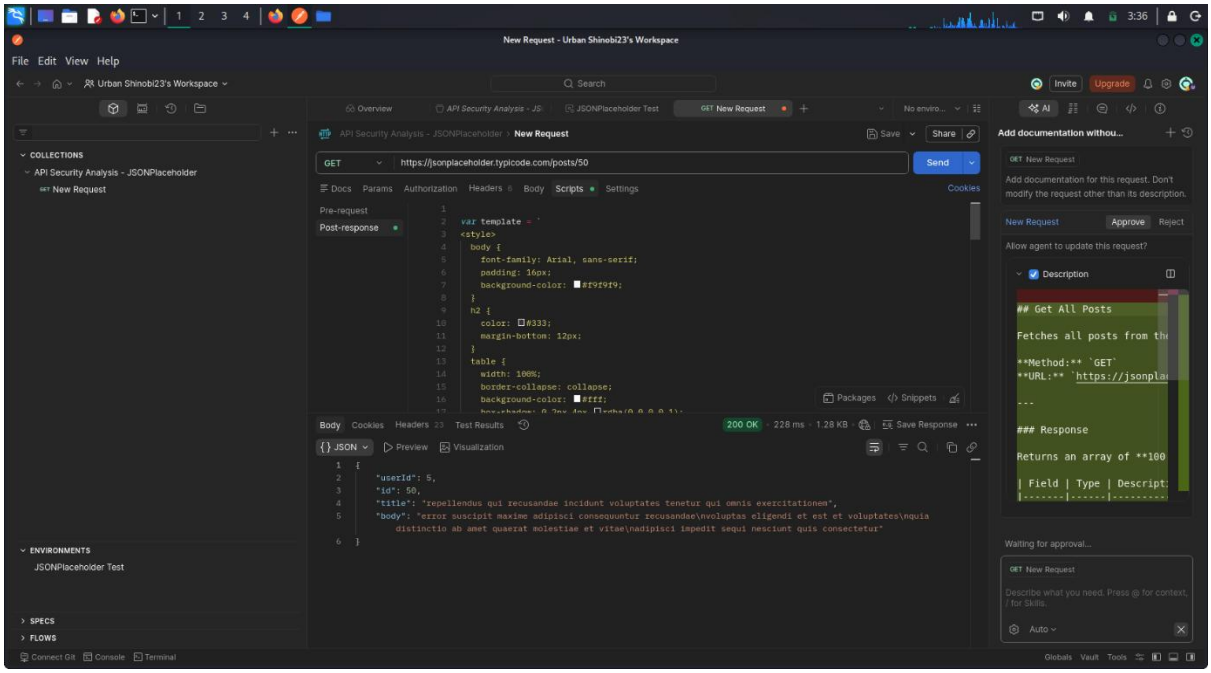
Test Performed

Two GET requests were sent to access different post IDs without any authentication or authorization.

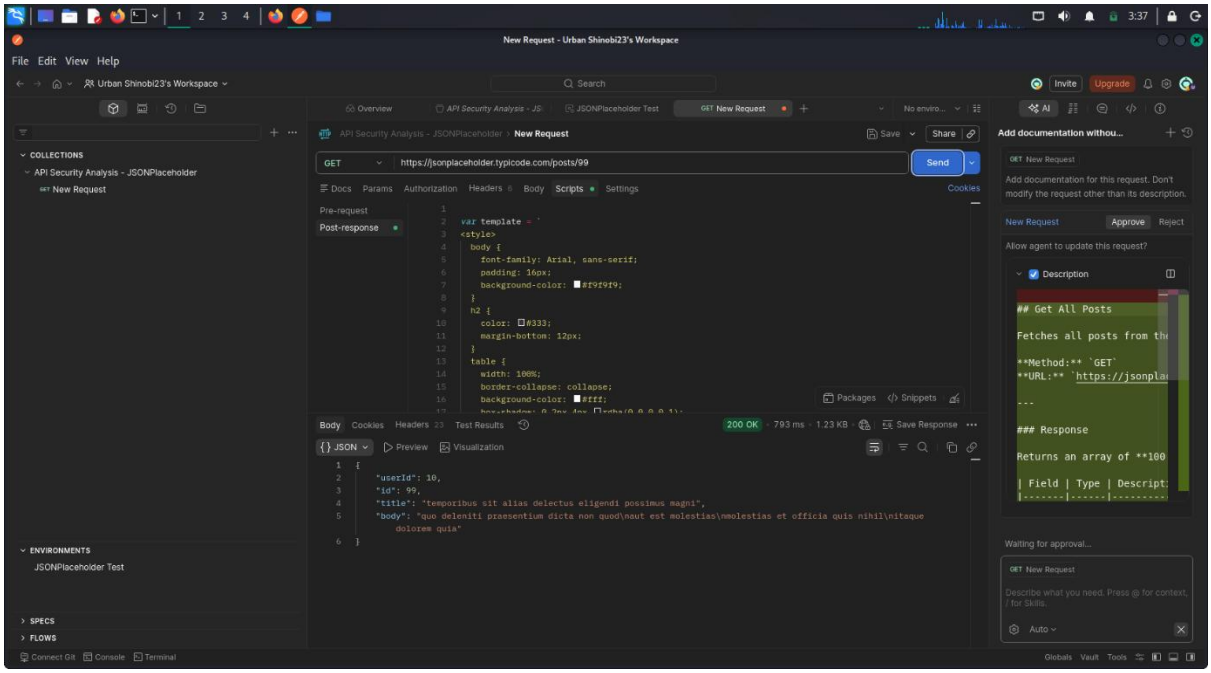
Evidence



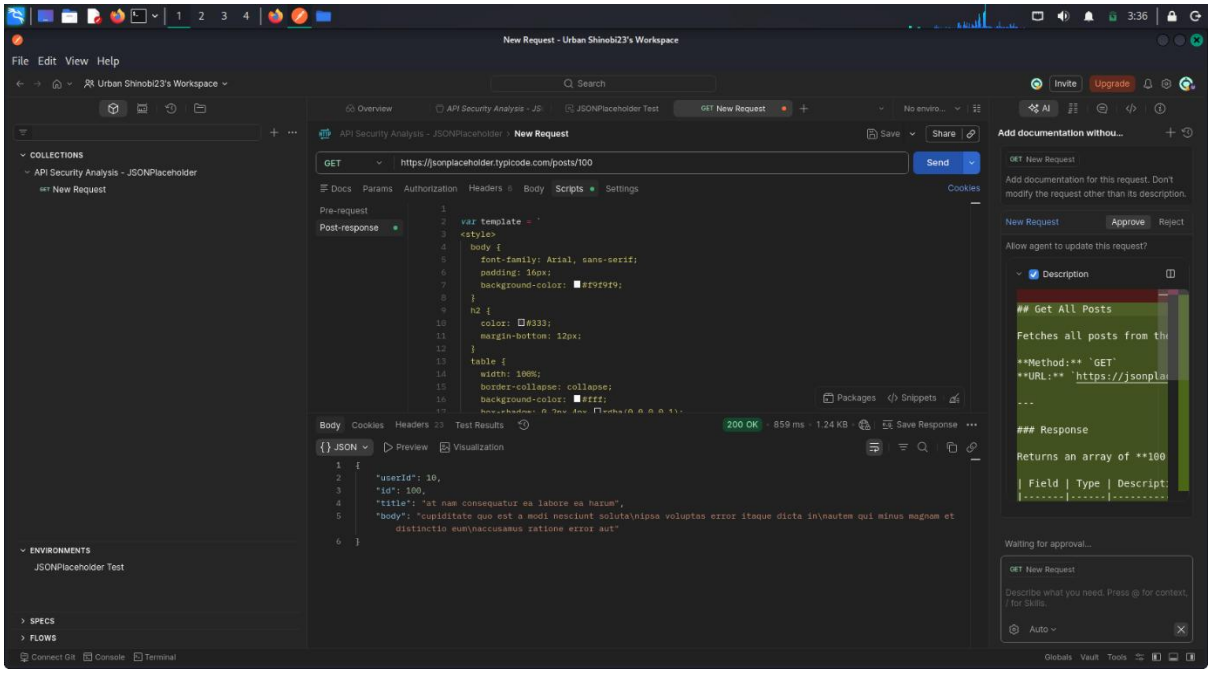
POST ID 1



POST ID 2



POST ID 3



POST ID 4

Business Impact

Impact Area	Description
Data Enumeration	Attackers can systematically access every record by iterating through IDs
Privacy Violation	Users can access other users' private posts, comments, and personal data
Competitive Intelligence	Competitors could extract all business data
Regulatory Fines	GDPR violations for unauthorized access to personal data can result in fines up to €20M

Remediation Steps

1. **Implement Authorization Checks**:
2. **Use UUIDs Instead of Sequential IDs**:
   - Replace predictable numeric IDs with UUIDs (e.g., 550e8400-e29b-41d4-a716-446655440000)
   - Makes guessing valid IDs exponentially harder
3. **Implement Access Control Lists (ACL)**:
   - Maintain explicit permissions for each user-resource relationship
   - Check permissions before returning any object
4. **Indirect Object References**:
   - Use maps that reference objects indirectly

- Never expose database keys directly to client

4.3 Excessive Data Exposure

Risk Identifier: DATA-001

Field	Description
Affected Endpoints	/users (primarily), /comments
OWASP Category	API3:2023 - Excessive Data Exposure
Severity	MEDIUM
Likelihood	Medium
Impact	Medium

Description

The API returns complete data objects that contain more information than necessary for typical client use cases. This is particularly evident in the /users endpoint, which exposes sensitive personal information that should be restricted.

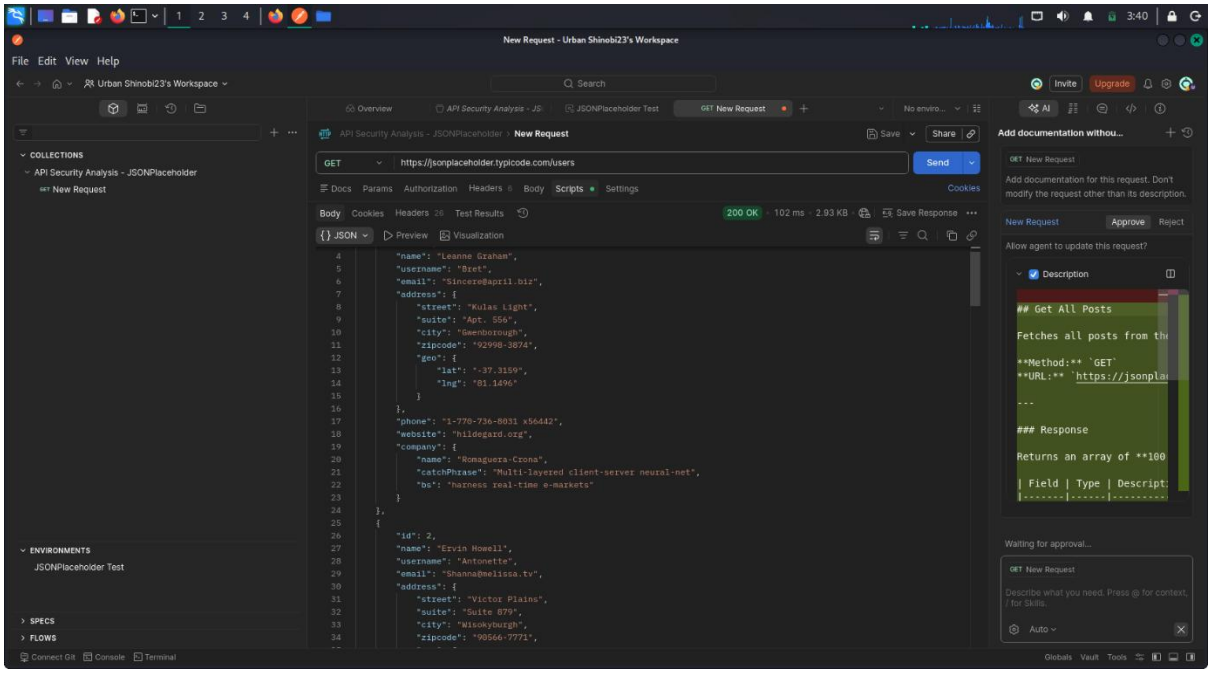
Test Performed

A GET request was sent to https://jsonplaceholder.typicode.com/users to examine all returned fields.

Excessive Data Identified

Field	Sensitivity	Why It's Excessive
email	HIGH	Personal identifier, spam risk, phishing target
phone	HIGH	Personal contact, telemarketing, SMS spam
address	HIGH	Physical location, privacy concern, stalking risk
geo (lat/lng)	HIGH	Precise location data, surveillance risk
website	LOW-MEDIUM	Personal/company website, may reveal other data

Evidence



Excessive data identified 1

Business Impact

Impact Area	Description
Privacy Compliance	Exposing email, phone, and address violates data minimization principles of GDPR
User Trust	Users expect their personal information to be protected
Spam/Phishing Risk	Exposed emails and phones make users targets for attacks
Competitive Intelligence	Competitors can scrape user lists and contact information

Remediation Steps

1. **Implement Data Transfer Objects (DTOs)**:
2. **Field Filtering Based on Permissions**:
    - Admin users might see all fields
    - Regular users see only public fields
    - Unauthenticated users see minimal fields
3. **GraphQL Consideration**: Consider GraphQL where clients specify exactly which fields they need

4. **Response Inspection**: Review all API responses for unnecessary data before deployment

4.4 Missing Rate Limiting

Risk Identifier: RATE-001

Field	Description
Affected Endpoints	ALL endpoints
OWASP Category	API4:2023 - Lack of Resources & Rate Limiting
Severity	CRITICAL
Likelihood	High
Impact	High

Description

The API implements no rate limiting controls whatsoever. An attacker can send hundreds or thousands of requests per second without any restriction, making the API highly vulnerable to denial of service attacks, web scraping, and brute force attempts.

Test Performed

Method 1: Manual Rapid Testing

- Sent 20 consecutive GET requests to /posts with zero delay
- All requests returned 200 OK
- No 429 (Too Many Requests) status codes observed

Method 2: Postman Collection Runner

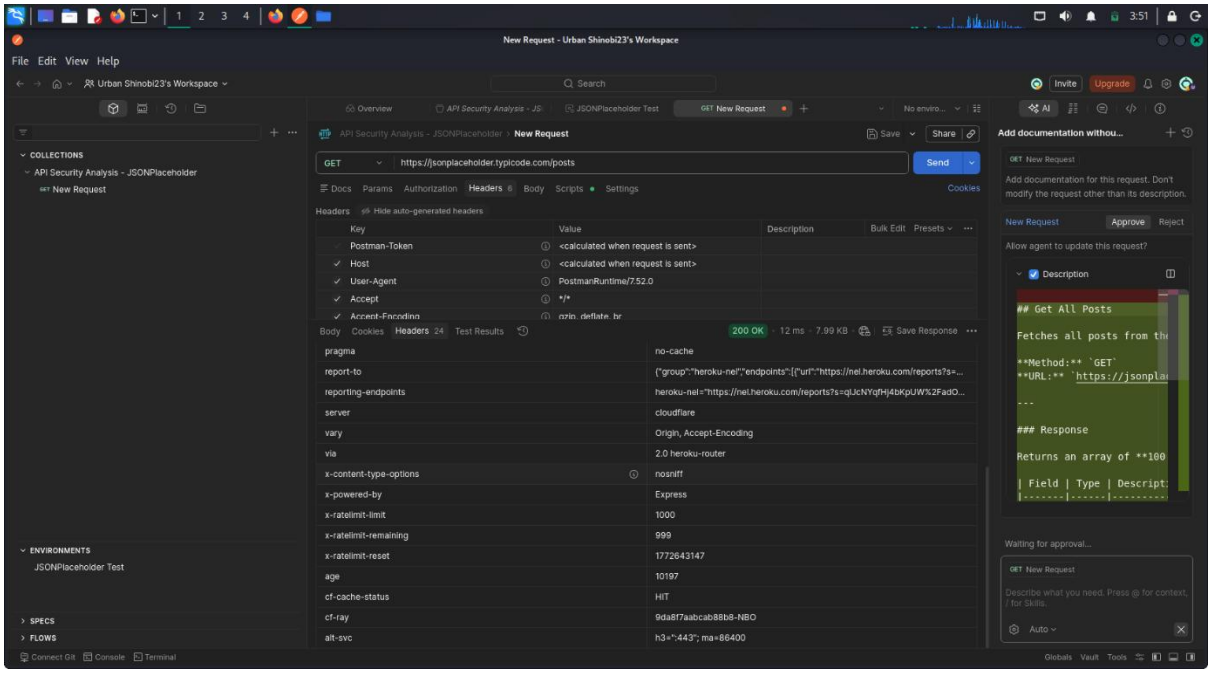
- Configured 50 iterations to /posts endpoint
- Zero delay between requests
- All 50 requests completed successfully in under 5 seconds

Test Results

Metric	Result
Total Requests	50
Time Elapsed	~4.3 seconds
Requests per Second	~11.6

Success Rate	100% (50/50)
429 Responses	0
Rate Limit Headers	None present

Evidence



RATE LIMITING 1

Business Impact

Impact Area	Description
Denial of Service	Attackers can overwhelm the API with requests, causing service degradation or outage
Data Scraping	Competitors or bad actors can extract the entire dataset in minutes
Brute Force Attacks	No protection against credential stuffing or password guessing
Infrastructure Costs	Uncontrolled request volume can dramatically increase cloud costs
Fair Usage	A single user can consume all resources, impacting other users

Remediation Steps

1. **Implement Rate Limiting Middleware**:
2. **Implement Tiered Rate Limits**:

User Type	Rate Limit	Rationale
Unauthenticated	60 requests/hour	Prevent anonymous abuse
Authenticated	1000 requests/hour	Legitimate users need more access
Premium	10000 requests/hour	Paid tier with higher limits
Admin	50000 requests/hour	Administrative functions

3. **Return Proper Headers**
4. **Advanced Rate Limiting Strategies**:
   - **Sliding Windows**: More accurate than fixed windows
   - **Token Bucket**: Allows burst traffic while maintaining average limits
   - **Distributed Rate Limiting**: Use Redis for rate limiting across multiple servers
   - **Endpoint-Specific Limits**: Stricter limits on expensive or sensitive endpoints
5. **Monitoring and Alerting**:
   - Log rate limit violations
   - Alert when IPs consistently hit limits
   - Consider temporary or permanent blocks for abusive IPs

5. Risk Summary Table

#	Risk	OWASP Category	Severity	Affected Endpoints	Remediation Priority
1	No Authentication	API2: Broken Authentication	CRITICAL	All	**IMMEDIATE**
2	No Rate Limiting	API4: Lack of Resources	CRITICAL	All	**IMMEDIATE**
3	Broken Object Level Authorization	API1: BOLA	HIGH	/{resource}/{id}	HIGH
4	Excessive Data Exposure	API3: Excessive Data	MEDIUM	/users	MEDIUM

---

6. Conclusion

6.1 Summary of Findings

This API Security Risk Analysis of jsonplaceholder.typicode.com has identified multiple critical vulnerabilities that would pose significant risks in a production environment:

1. **Critical Risks (2)**: Complete lack of authentication and rate limiting make the API completely open to abuse, data scraping, and denial of service attacks.
2. **High Risk (1)**: Broken Object Level Authorization allows attackers to access any record by simply guessing or enumerating IDs.
3. **Medium Risk (1)**: Excessive data exposure leaks sensitive user information that should be protected.

6.2 Key Recommendations

- **Immediate Actions (Priority 1):**

1. Implement authentication for all endpoints
2. Deploy rate limiting across the entire API

- **Short-term Actions (Priority 2):**
  3. Add authorization checks to verify users can only access their own resource
  4. Replace sequential IDs with UUIDs

- **Medium-term Actions (Priority 3):**
  5. Implement DTOs to control data exposure
  6. Add security headers to all responses

6.3 Final Risk Assessment

While JSONPlaceholder serves its purpose as a free testing API, this analysis demonstrates why production APIs require robust security controls. The vulnerabilities identified here would be unacceptable in any commercial SaaS application handling real user data.

Overall Security Posture: UNSAFE FOR PRODUCTION USE

Appendix A: OWASP API Security Top 10 Reference

Category	Description	Found in This Assessment
API1:2023	Broken Object Level Authorization	✓ (Section 4.2)
API2:2023	Broken Authentication	✓ (Section 4.1)
API3:2023	Broken Object Property Level Authorization	Not tested
API4:2023	Unrestricted Resource Consumption	✓ (Section 4.4)
API5:2023	Broken Function Level Authorization	Not tested
API6:2023	Unrestricted Access to Sensitive Business Flows	Not tested
API7:2023	Server Side Request Forgery	Not tested
API8:2023	Security Misconfiguration	Partial (headers)
API9:2023	Improper Inventory Management	Not tested
API10:2023	Unsafe Consumption of APIs	Not tested